Step 2 — A Real Terminal Chat With Memory and Live Streaming — Solution

> Back to this step · Hint 1 · Hint 2 · Solution

Story — Step 2: the model remembers nothing between calls (Doc03). So the app keeps a growing `history` list and resends it every turn — that list *is* the memory. Streaming makes the reply appear as it's written. Trimming keeps a long chat under the model's limit. **If not:** every reply would forget the previous one, the user would stare at a blank line while the whole reply is generated, and a long chat would eventually crash with a 400 error.

After this step the project is a working chat app. Files from Step 1 that aren't shown here (`config.py`, `exceptions.py`, `utils/logger.py`) don't change.

Solution — prompt file, token budget, and a unit test

Changed from Step 1: one new function at the bottom, `stream_message()`. Everything above it is Step 1's code, unchanged.

Story — `models/llm.py`: Step 1's file plus Doc04's `stream_message()`. The one change from Doc04 is `flush=True`. **If not:** the terminal holds printed text back until the line ends, so the "stream" would appear all at once at the end.

```
# src/models/llm.py
# The only file that talks to the OpenAI SDK: create_client(), the startup
# key check, send_message(), and stream_message() for live replies.
import openai
from openai import OpenAI

from src.config import Config

MODEL = "gpt-4o-mini"

def create_client(config: Config) -> OpenAI:
    return OpenAI(api_key=config.openai_api_key)

def check_api_key_at_startup(client: OpenAI) -> bool:
    try:
        client.chat.completions.create(
```

```
            model=MODEL,
            messages=[{"role": "user", "content": "ping"}],
            max_tokens=1,
        )
    except openai.AuthenticationError:
        return False
    return True

def send_message(client: OpenAI, history: list[dict], user_input: str) -> str:
    history.append({"role": "user", "content": user_input})
    response = client.chat.completions.create(model=MODEL, messages=history)
    reply = response.choices[0].message.content
    history.append({"role": "assistant", "content": reply})
    return reply

# new in Step 2: everything below this line
def stream_message(
    client: OpenAI, history: list[dict], user_input: str
) -> str:
    history.append({"role": "user", "content": user_input})
    stream = client.chat.completions.create(
        model=MODEL, messages=history, stream=True
    )
    full_reply = ""
    for chunk in stream:
        piece = chunk.choices[0].delta.content
        if piece:
            print(piece, end="", flush=True)
            full_reply = full_reply + piece
    print()
    history.append({"role": "assistant", "content": full_reply})
    return full_reply
```

Story — `prompts/concierge.txt` **:** the assistant's personality, as plain text. (It's called "concierge" already, because in Step 4 this chat becomes the Concierge agent.) **If not:** changing the tone would mean editing Python code.

`src/prompts/concierge.txt`

```
You are the SupportDesk concierge. Be friendly and brief.
Answer general questions and small talk about the service.
```

Story — `utils/prompts.py` **:** one function that reads a prompt file by name, so no agent opens files itself. **If not:** every agent would repeat the same `open(...)` code, each with its own copy of the folder path.

```
# src/utils/prompts.py
# load_prompt(name): reads src/prompts/<name>.txt, so prompts live
# in text files instead of Python strings.

PROMPTS_DIR = "src/prompts"

def load_prompt(name: str) -> str:
    path = PROMPTS_DIR + "/" + name + ".txt"
    with open(path) as prompt_file:
        return prompt_file.read().strip()
```

Story — `utils/history.py`: estimates the size of the history (Doc03's "about 4 characters per token") and drops the oldest turns until it fits a budget. The system prompt always stays. **If not:** a long chat would hit the context limit and fail with a 400 error mid-conversation — or, if you dropped from the front blindly, the assistant would lose its personality.

```
# src/utils/history.py
# Keeps a chat under the model's context limit: estimate_tokens() guesses
# its size (about 4 characters per token), and trim_history() drops the
# oldest turns — never the system prompt.

CHARS_PER_TOKEN = 4

def estimate_tokens(history: list[dict]) -> int:
    total_chars = 0
    for message in history:
        total_chars = total_chars + len(message["content"])
    return total_chars // CHARS_PER_TOKEN

def trim_history(history: list[dict], max_tokens: int) -> list[dict]:
    while estimate_tokens(history) > max_tokens and len(history) > 2:
        del history[1]
    return history
```

Changed from Step 1: the startup part of `main()` is the same. What's new: 2 imports, `MAX_HISTORY_TOKENS`, and the loop that replaces the single `send_message()` call.

Story — `main.py`: Step 1's startup, then a loop. It skips empty lines, trims before every call, and loads the system prompt from its file. **If not:** an empty Enter would cost an API call, and nothing would stop the history from growing past the limit.

```
# main.py
# The file you run: python main.py
# Loads the settings, checks the key, then runs a chat loop: each reply
```

```
# streams live, and the history is trimmed before every call.
import sys

from src.config import load_config
from src.exceptions import MissingConfigError
from src.models.llm import (
    check_api_key_at_startup,
    create_client,
    stream_message,
)
# new in Step 2: trim_history and load_prompt
from src.utils.history import trim_history
from src.utils.logger import get_logger
from src.utils.prompts import load_prompt

# new in Step 2
MAX_HISTORY_TOKENS = 3000

def main() -> None:
    try:
        config = load_config()
    except MissingConfigError as e:
        print(f"Setup problem: {e}")
        sys.exit(1)

    logger = get_logger(__name__)
    client = create_client(config)
    if not check_api_key_at_startup(client):
        logger.error("Authentication failed — check OPENAI_API_KEY.")
        print("Login failed — check OPENAI_API_KEY in your .env file.")
        sys.exit(1)

    # new in Step 2: from here down — the loop replaces the one fixed call
    history = [{"role": "system", "content": load_prompt("concierge")}]
    print("SupportDesk AI — type a message, or /quit to exit.")

    while True:
        message = input("You: ").strip()
        if message == "/quit":
            break
        if message == "":
            continue

        trim_history(history, MAX_HISTORY_TOKENS)
        logger.debug(f"Sending {len(history)} earlier messages")
        print("Assistant: ", end="", flush=True)
        stream_message(client, history, message)
```

```
if __name__ == "__main__":
    main()
```

Story — `tests/unit/test_history.py`: proves trimming keeps the system prompt and the newest turn, with no API key and no network. Each case prints `OK` or `WRONG`, like the Step 1 tests. **If not:** a trimming bug would only show up as a strange reply after a long chat — very hard to trace back here.

```
# tests/unit/test_history.py
# Checks the history helpers. Each case prints OK or WRONG.
# No API calls, no API key needed.
# Run from the repo root: python -m tests.unit.test_history
from src.utils.history import estimate_tokens, trim_history


def test_estimate_tokens() -> None:
    history = [
        {"role": "system", "content": "a" * 20},
        {"role": "user", "content": "b" * 20},
    ]
    result = "OK"
    if estimate_tokens(history) != 10:
        result = "WRONG"
    print(f"case 1 (40 characters = 10 tokens) -> {result}")


def test_short_history_untouched() -> None:
    history = [
        {"role": "system", "content": "be nice"},
        {"role": "user", "content": "hi"},
    ]
    trim_history(history, max_tokens=100)
    result = "OK"
    if len(history) != 2:
        result = "WRONG"
    print(f"case 2 (short chat not trimmed)    -> {result}")


def test_long_history_trimmed() -> None:
    history = [{"role": "system", "content": "be nice"}]
    for number in range(1, 6):
        history.append({"role": "user", "content": str(number) * 400})
    trim_history(history, max_tokens=250)
    result = "OK"
    if history[0]["role"] != "system":
```

```
        result = "WRONG"
    if history[-1]["content"] != "5" * 400:
        result = "WRONG"
    print(f"case 3 (long chat trimmed)         -> {result}")


if __name__ == "__main__":
    test_estimate_tokens()
    test_short_history_untouched()
    test_long_history_trimmed()
```

Run the test: `python -m tests.unit.test_history`

```
case 1 (40 characters = 10 tokens) -> OK
case 2 (short chat not trimmed)    -> OK
case 3 (long chat trimmed)         -> OK
```

Case 3 checks the two things that matter: the system prompt is still first, and the newest turn (turn 5) is still last. (Turns 4 and 5 stay — about 200 tokens; adding turn 3 would pass 250.)

Changed from Step 1: `tests/integration/test_live_agents.py` gets case 3 — one new function and one new call.

Story — `tests/integration/test_live_agents.py`: memory is this step's feature, so it gets a real check: turn 3 must use what was said in turn 1. **If not:** a bug that forgets the history would only show up when a user complained.

```
# tests/integration/test_live_agents.py
# Checks the app against the real OpenAI API — each step adds cases.
# Each case prints OK or WRONG. Needs a valid OPENAI_API_KEY in .env.
# Run from the repo root: python -m tests.integration.test_live_agents
from openai import OpenAI

from src.config import load_config
from src.models.llm import (
    check_api_key_at_startup,
    create_client,
    send_message,
)


def test_first_reply(client: OpenAI) -> None:
    reply = send_message(client, [], "What's 2+2?")
    result = "OK"
    if "4" not in reply:
```

```
        result = "WRONG"
    print(f"case 1 (first reply) -> {result}: {reply}")


def test_bad_key() -> None:
    bad_client = OpenAI(api_key="sk-fake-key-123")
    result = "OK"
    if check_api_key_at_startup(bad_client):
        result = "WRONG"
    print(f"case 2 (bad key)     -> {result}")


# new in Step 2
def test_memory(client: OpenAI) -> None:
    history = [{"role": "system", "content": "You are a helpful assistant."}]
    send_message(client, history, "My order number is 4471.")
    send_message(client, history, "Thanks, that's all for now.")
    reply = send_message(client, history, "What was my order number?")
    result = "OK"
    if "4471" not in reply:
        result = "WRONG"
    print(f"case 3 (memory)      -> {result}: {reply}")


if __name__ == "__main__":
    config = load_config()
    client = create_client(config)
    test_first_reply(client)
    test_bad_key()
    # new in Step 2
    test_memory(client)
```

Run the tests: `python -m tests.integration.test_live_agents` — the Step 1 cases, then:

```
case 3 (memory)      -> OK: Your order number was 4471.
```

Run the app: `python main.py`, then:

```
SupportDesk AI — type a message, or /quit to exit.
You: my name is Ali
Assistant: Nice to meet you, Ali! How can I help you today?
You: what's the capital of France?
Assistant: The capital of France is Paris.
You: what's my name?
```

```
Assistant: Your name is Ali.
You: /quit
```